Intermediate (swap in a structured parser) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `structured_output_practice.py`: `.with_structured_output(Model)` wraps a chat model so that, instead of returning free text, it returns a filled-in instance of your Pydantic `Model` — LangChain tells the model the exact shape to produce and parses the reply back into typed Python for you. It's the same constrained-decoding idea you met in Doc04 with the raw SDK's `.parse()`, just the LangChain way of asking for it. This exercise is where you prove to yourself the two are the same tool — including the same trap: if a field is *required*, the model has no way to say "I don't know," so it will invent a plausible-looking value rather than leave it blank. Making a field optional (`str | None`) is what lets the model honestly return nothing. **If not:** the Build Task's `model.with_structured_output(ExtractedInfo)` would be the first time you met LangChain's structured-output call.

Solution

```
# structured_output_practice.py
from pydantic import BaseModel, ValidationError
from langchain_openai import ChatOpenAI


class Person(BaseModel):
    # str | None, not str — a required field would force the model to
    # invent a value when the text has none (Doc04's rule).
    name: str | None
    age: int | None


# .with_structured_output(Person) is LangChain's version of Doc04's
# .parse(). This returns a Person, or None when nothing was found.
def extract_person(text):
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    structured_model = model.with_structured_output(Person)
    try:
        person = structured_model.invoke(text)
    except ValidationError:
        return None          # a bad reply gives None, not a crash
```

```
    if person.name is None and person.age is None:
        return None          # nothing found -> "no person", not all-None
    return person


# Try it on a text with a person, and one without.
texts = [
    "Extract: Maria is 34 years old.",
    "Extract: the sky is blue today.",
]
for text in texts:
    print(f"{text} -> {extract_person(text)}")
```

Expected output:

```
Extract: Maria is 34 years old. -> name='Maria' age=34
Extract: the sky is blue today. -> None
```

Note: this is the same pattern as Doc04's `extract_info()`, now through LangChain — it's what the Build Task's chain relies on.